

PySpark e Apache Kafka para processamento de dados em lote e streaming

81%

TrilhaFórum

Projeto 7 - Executando o Real-Time ETL Stack com Airflow, Kafka, PySpark e Cassandra - Parte 1/3

video03:29

Projeto 7 - Executando o Real-Time ETL Stack com Airflow, Kafka, PySpark e Cassandra - Parte 2/3

video02:20

Projeto 7 - Executando o Real-Time ETL Stack com Airflow, Kafka, PySpark e Cassandra - Parte 3/3

video04:05

Projeto 7 - Conclusão

video03:52

Otimizações de Desempenho no Stack PySpark-Kafka

videolivro01:52

Monitoramento de Pipelines em Tempo Real

videolivro02:18

Arquivos Usados Neste Capítulo

pdf

Bibliografia, Referências e Links Úteis

pdf



Otimizações de Desempenho no Stack PySpark-Kafka

A superação do desempenho em uma pilha que utiliza PySpark e Kafka envolve práticas específicas para garantir a eficiência no processamento e na transmissão de dados. Alguns pontos-chave incluem:

Não Kafka

Ajuste do tamanho dos lotes: Configure o parâmetro `batch.size` para um valor adequado ao seu volume de dados e recursos disponíveis. Isso ajuda a equilibrar latência e taxa de transferência.

Particionamento eficiente: Garanta que os descritos sejam particionados específicos para distribuição de carga entre os consumidores.

Retenção de mensagens: Configure a retenção (`retention.ms` ou `retention.bytes`) de acordo com as necessidades do pipeline, evitando sobrecarga desnecessária no

?

Suporte

?

Suporte



cluster.

Ajuste do `fetch.max.bytes`: Modifique para evitar sobrecarregar o consumidor, especialmente em pipelines de alta latência.

Sem PySpark

Persistência adequada: Utilize os métodos `cache()` ou `persist()` para armazenar dados intermediários, mas apenas quando necessário, evitando armazenar RDDs/Datasets desnecessariamente.

Gerenciamento de particionamento: Ajuste o número de partições no RDD ou `DataFrame` para evitar subutilização de recursos ou sobrecarga. Isso pode ser feito com `repartition()` ou `coalesce()`.

Evite ações desnecessárias: Minimize operações como `count()`, `collect()` e `show()`, que podem forçar a execução prematura de um pipeline.

Uso de APIs nativas: Prefira APIs estruturadas (`DataFrame` ou `Dataset`) em vez de RDDs, pois são otimizadas pelo Catalyst Optimizer.

Integração PySpark-Kafka

Batch Size e Frequência: Sem Streaming Estruturado, ajuste as configurações `maxOffsetsPerTrigger` e `trigger` para controlar o volume de dados processados por microbatch, evitando latência ou gargalos.

Checkpointing: Configure um diretório de checkpoint para permitir a recuperação em caso de falhas e garantir consistência.

Gerenciamento de Offset: Utilize os offsets iniciais e finais de forma estratégica, especialmente para lidar com histórico de dados ou para evitar sobrecarga inicial.

Geral

Monitoramento: Utilize ferramentas como Kafka Manager, Spark UI e Control Center para monitorar gargalos em tempo real.

Recursos Alocados: Ajuste configurações como `executorMemory`, `executorCores` e o número de executores para atender à carga de trabalho.

Compactação: No Kafka, use atualizações de mensagens (por exemplo, `snappy` ou `lz4`) para reduzir o uso de largura de banda e armazenamento.

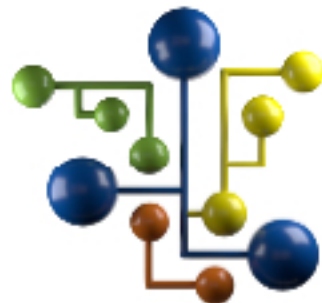
Uma combinação dessas práticas garante melhor uso dos recursos, escalabilidade e desempenho otimizado em pipelines construídos com PySpark e Kafka.

?

Suporte

?

Suporte



Equipe DSA

Continue trilhando uma excelente jornada de aprendizagem.

?

Suporte

?

Suporte



?

Suporte

?

Suporte

